

Aula 04 — Métodos Não Paramétricos (KNN)

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §4.1–4.5; ISLP §3.5 e cap. 7

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- explicar o que distingue um método *não paramétrico* de um paramétrico e por que ele troca viés por variância;
- descrever a estratégia de *bases* (séries ortogonais, *splines*) para flexibilizar a regressão linear;
- definir o estimador de k **vizinhos mais próximos** (KNN) e o papel de k no balanço viés–variância;
- enxergar todos esses métodos como *médias locais* / *suavizadores lineares*.

Esta aula é uma mudança de paradigma em relação às Aulas 01–02. Lá fixávamos a forma de r — linear, polinomial de grau p — e estimávamos poucos parâmetros. A pergunta desta aula é a consequência natural:

e se eu não quiser supor forma alguma para $r(\mathbf{x})$?

A resposta é deixar os *dados* ditarem a forma. O preço, você já sabe qual é — e a Aula 05 vai mostrar que ele é bem mais alto do que parece quando p cresce.

1 O que é um método não paramétrico?

Um método **paramétrico** supõe que $r(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$ pertence a uma família indexada por um número *fixo e finito* de parâmetros (ex.: regressão linear, $p + 1$ coeficientes). Um método **não paramétrico** não impõe uma forma funcional fixa: a complexidade efetiva do modelo *cresce com n* . A suposição é mais fraca — tipicamente apenas que r é *suave*.

Atenção: “não paramétrico” não quer dizer “sem parâmetros”

O nome engana. A diferença entre as duas famílias não está em *quantos* parâmetros o método usa — está em *quando* esse número fica decidido.

No paramétrico, fica decidido **antes de você olhar os dados**: uma regressão linear com p covariáveis tem $p + 1$ coeficientes, e terá $p + 1$ tanto com 50 observações quanto com 50 milhões. Mais dados estimam melhor esses coeficientes, mas não ampliam o repertório do modelo — ele continua só conseguindo desenhar hiperplanos.

No não paramétrico, não fica. Tome o KNN com $k = 5$ *fixo*: com 50 observações ele gasta o equivalente a dez parâmetros; com 50 mil, dez mil — o número de parâmetros efetivos é n/k , conta que você fará na **Lista teórica 04**. Mesmo método, mesmo k , e um repertório incomparavelmente maior, porque cada vizinhança ficou mais estreita e há muito mais delas. É isso que “a complexidade efetiva cresce com n ” quer dizer.

Ideia central

Em relação aos paramétricos, métodos não paramétricos **reduzem o viés** (capturam relações arbitrárias) ao custo de **aumentar a variância**. Continuamos no mundo da Aula 01: o segredo é controlar a flexibilidade por um *tuning parameter* (o k do KNN, o número de nós de um *spline*), escolhido por validação cruzada (Aula 03).

Observação 1.1. Há uma ponte importante com a Aula 02. Ridge, Lasso e os métodos desta aula perseguem o mesmo alvo — o ponto certo do balanço viés-variância —, mas chegam a ele em sentidos contrários, porque atacam parcelas diferentes.

A penalização é para o modelo *flexível demais para a amostra*: o MQO com muitas covariáveis erra por **variância**, e encolher os coeficientes troca um pouco de viés por bastante variância a menos. Os métodos desta aula são para o caso inverso, o modelo *rígido demais para a verdade*: se r não é linear, nenhum volume de dados conserta uma reta — o erro que sobra é **viés**, e a única saída é abrir mão da forma funcional, pagando em variância.

2 Estratégia de bases: séries ortogonais e *splines*

A primeira ideia para ganhar flexibilidade sem sair do arcabouço linear é **ampliar o conjunto de atributos** com funções $\phi_1, \dots, \phi_I$ das covariáveis e ajustar

$$g(x) = \sum_{j=1}^I \beta_j \phi_j(x),$$

que é *linear nos* β_j — estimável por mínimos quadrados (Aula 02). Aqui a flexibilidade é controlada por I (número de funções da base).

2.1 Séries ortogonais

Usa-se uma base ortonormal de L^2 (Fourier, polinômios ortogonais, *wavelets*). Aproxima-se r por suas primeiras I componentes na base; I é o *tuning parameter* (mais termos $\Rightarrow$ mais flexível). É a ideia por trás de “regressão polinomial” vista na Aula 01 (a base dos monômios $1, x, x^2, \dots$).

2.2 *Splines*

Splines são funções polinomiais por partes, “coladas” suavemente em pontos chamados **nós** (*knots*) $t_1, \dots, t_p$. Uma base conveniente para *splines* de grau k é

$$f_1(x) = 1, f_2(x) = x, \dots, f_{k+1}(x) = x^k, \quad f_{k+1+j}(x) = (x - t_j)_+^k, \quad j = 1, \dots, p,$$

onde $(u)_+ = \max(u, 0)$. Com isso $I = k + 1 + p$, e novamente estimam-se os coeficientes por mínimos quadrados.

Ideia central: por que a base truncada funciona

Vale entender por que $(x - t_j)_+^k$ é a peça certa. Essa função é identicamente nula à esquerda de t_j e um polinômio de grau k à direita. Somá-la ao modelo, portanto, *não muda nada antes do nó* e permite mudar a curvatura *depois* dele — e como ela e suas $k - 1$ primeiras derivadas se anulam em t_j , a emenda sai suave de graça. Cada nó compra exatamente um grau de liberdade a mais, localizado onde você quiser.

Variantes importantes: *B-splines* (base numericamente estável, é o que as bibliotecas usam de fato), *natural splines* (impõem linearidade fora dos nós extremos, reduzindo a variância na

fronteira) e *smoothing splines* (em vez de escolher nós, põem um nó em cada observação e penalizam a curvatura $\int (g''(x))^2 dx$ — caso particular de RKHS, §4.6 do [AME]).

Poucos nós dão uma curva rígida (subajuste); muitos nós dão uma curva nervosa (superajuste). O número e a posição dos nós são o botão de complexidade — é a Aula 01 de novo, com outro nome.

3 k vizinhos mais próximos (KNN)

O KNN é talvez o método não paramétrico mais intuitivo: para prever em $\mathbf{x}$, faça a *média das respostas dos k pontos de treino mais próximos de $\mathbf{x}$* .

Definição 3.1 (Regressão KNN). Seja $\mathcal{N}_{\mathbf{x}}$ o conjunto dos índices das k observações de treino mais próximas de $\mathbf{x}$ (segundo uma distância p , em geral euclidiana): $\mathcal{N}_{\mathbf{x}} = \{i : d(\mathbf{X}_i, \mathbf{x}) \leq d_{\mathbf{x}}^k\}$, onde $d_{\mathbf{x}}^k$ é a distância do k -ésimo vizinho. Então

$$\hat{r}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_{\mathbf{x}}} Y_i. \quad (1)$$

Repare no que essa fórmula *não* tem: nenhum parâmetro a estimar, nenhuma otimização, nenhum ajuste. O “treino” do KNN consiste em guardar os dados. Todo o trabalho acontece na hora da predição — por isso o método é chamado de *preguiçoso* (*lazy learner*).

O parâmetro k controla a flexibilidade:

- k **pequeno** (ex.: $k = 1$): vizinhança minúscula, segue cada ponto $\Rightarrow$ *viés baixo, variância alta* (curva “nervosa”, superajuste).
- k **grande**: média sobre muitos pontos; quando $k \rightarrow n$, $\hat{r}$ tende à média global (constante) $\Rightarrow$ *viés alto, variância baixa* (subajuste).

Escolhe-se k por validação cruzada.

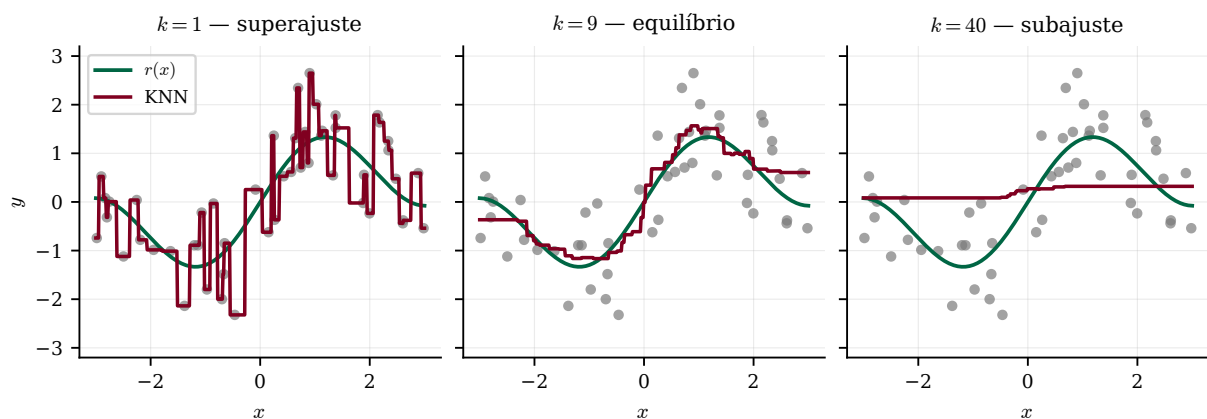


Figura 1: KNN sobre a mesma amostra de $n = 50$ pontos da Aula 01. Com $k = 1$ a estimativa passa por cima de cada observação e vira uma escada — é a versão KNN do polinômio de grau 15. Com $k = 40$, quase toda a amostra entra em cada média e a curva quase não distingue um x do outro. O k faz exatamente o papel que o grau do polinômio fazia, só que *ao contrário*: aqui é o *k pequeno* que significa flexível.

Atenção

KNN exige uma noção de distância e, portanto, **depende fortemente da escala** das covariáveis. Se uma variável está em reais e outra em milhares de reais, a segunda domina

completamente a distância euclidiana e a primeira poderia nem existir. *Padronize* antes (Aula 07). Além disso, o KNN sofre da *maldição da dimensionalidade*: em p alto, “vizinhos” deixam de ser próximos — é o tema inteiro da Aula 05.

4 Uma visão unificadora: suavizadores lineares

KNN, *séries* e *splines* são todos **suavizadores lineares**: a predição é uma combinação linear das respostas,

$$\hat{r}(\mathbf{x}) = \sum_{i=1}^n \ell_i(\mathbf{x}) Y_i,$$

com pesos $\ell_i(\mathbf{x})$ que dependem só das covariáveis (não de $\mathbf{Y}$). Para o KNN, $\ell_i(\mathbf{x}) = \frac{1}{k} \mathbf{1}\{i \in \mathcal{N}_{\mathbf{x}}\}$; para uma base de I funções, $\ell_i(\mathbf{x})$ vem do ajuste de mínimos quadrados na base.

Ideia central

Vista assim, a diferença entre os métodos desta aula é só *o formato dos pesos*: o KNN concentra todo o peso numa janela retangular em torno de $\mathbf{x}$, larga onde há poucos dados e estreita onde há muitos; uma base de *splines* espalha o peso segundo as funções da base. Essa estrutura comum explica por que todos enfrentam o mesmo dilema — o botão de flexibilidade $\leftrightarrow$ viés–variância — e por que valem para todos os atalhos vistos antes, como o do LOOCV via *leverage* (Aula 03, Prop. 3.1): basta que $\ell_i(\mathbf{x})$ não dependa de $\mathbf{Y}$.

Resumo

- Métodos não paramétricos não fixam a forma de r ; reduzem viés e aumentam variância em relação aos paramétricos. A complexidade cresce com n .
- *Bases* (séries, *splines*): linearizam o problema; flexibilidade via número de termos/nós.
- **KNN (1)**: média dos k vizinhos; k controla viés–variância, com k *pequeno* = flexível (Fig. 1).
- Todos são *suavizadores lineares*; *padronize* as covariáveis e cuidado com a dimensão (Aula 05).

Leitura recomendada. [AME] §4.1–4.2 (séries e *splines*) e §4.3 (KNN, com a Figura 4.2 sobre o efeito de k). [ISLP] §3.5 (regressão linear $\times$ KNN, onde fica claro quando o paramétrico ainda ganha) e Capítulo 7 (*Moving Beyond Linearity*), em especial §7.4–7.5 sobre *splines*.

Para praticar. `Lista teorica 04.pdf` (teórica, com gabarito) e `Lista prática 04.ipynb` (prática, para completar as lacunas), nesta mesma pasta.